Chương 14

Ứng dụng của toán học trong công nghệ thông tin

Sinh viên khoa Công nghệ thông tin thường đặt câu hỏi: *"Tại sao chúng ta phải học đạo hàm, tích phân, ma trận hay không gian vector...? Trong khi lập trình chỉ cần logic và cấu trúc dữ liệu?"*. Câu trả lời là: Lập trình giúp chúng ta tạo ra ứng dụng, nhưng toán học mới là chìa khóa để chúng ta tạo ra những hệ thống cốt lõi như trí tuệ nhân tạo (AI), đồ họa 3D, Search Engine hay thuật toán mã hóa... Chương này sẽ trình bày một số ứng dụng thực tế sinh động nhất của toán học trong khoa học máy tính.

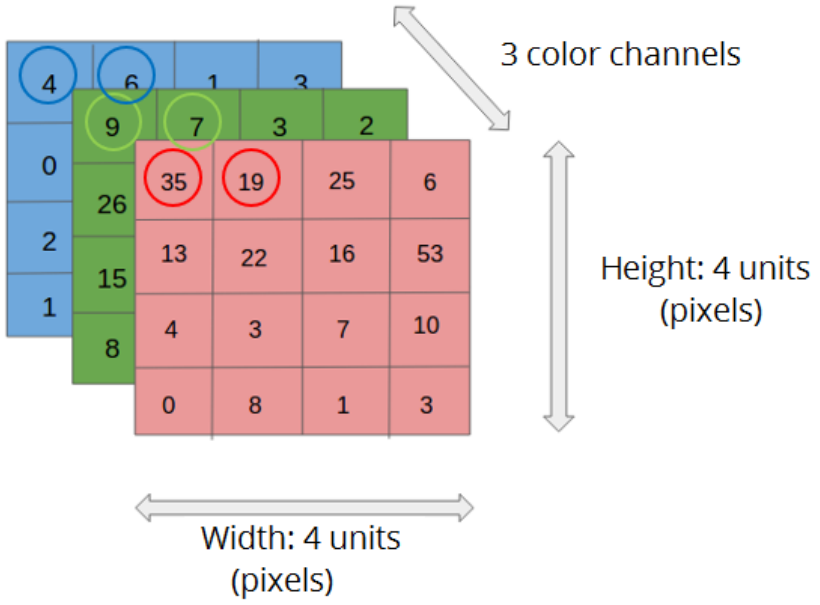
14.1 Ma trận trong xử lý ảnh (Thị giác máy tính)

Bản chất của hình ảnh và phép tích chập

Những tấm ảnh bạn nhìn thấy trên màn hình máy tính bản chất chính là một **ma trận**. Đối với ảnh màu, nó là một tensor gồm 3 ma trận chồng lên nhau đại diện cho 3 kênh màu RGB (đỏ, xanh lá, xanh lam). Mỗi phần tử (pixel) có giá trị từ $0 - 255$.

Bộ lọc Instagram hoạt động như thế nào? Khi bạn áp dụng một filter (bộ lọc) lên ảnh, máy tính không làm gì khác ngoài việc sử dụng **phép tích chập tuyến tính**. Nếu gọi ảnh gốc là ma trận I và bộ lọc là ma trận hạt nhân K, ma trận ảnh mới sẽ được tính bằng cách trượt ma trận K dọc theo ma trận I và tính tổng các tích tương ứng.

Trong học máy, các phép nhân, xoay, lật ma trận còn được dùng làm kỹ thuật Data Augmentation (Tăng cường dữ liệu) để huấn luyện AI.



Hình 14.1: Ma trận các pixel với 3 kênh màu RGB (đại diện cho màu đỏ, xanh lá, xanh lam)

Ví dụ 14.1.1 Dùng ma trận 3×3 (Filter phát hiện cạnh - Edge Detection) để quét qua một ma trận ảnh đơn giản bằng thư viện SciPy.

```
import numpy as np
from scipy.signal import convolve2d

# 1. Ma tran bieu dien mot buc anh don gian (5x5)
# Giong nhu mot vung sang (50) nam giua nen toi (10)
image = np.array([
[10, 10, 10, 10, 10],
[10, 10, 50, 10, 10],
[10, 50, 50, 50, 10],
[10, 10, 50, 10, 10],
[10, 10, 10, 10, 10]
])

# 2. Ma tran Kernel (Filter phat hien vien/canh - Edge Detection)
kernel = np.array([
[-1, -1, -1],
[-1,  8, -1],
[-1, -1, -1]
])

# 3. Thuc hien phep tich chap (Convolution)
result_image = convolve2d(image, kernel, mode='valid')

print("Anh sau khi ap dung Filter:")
print(result_image)
# Output se lam noi bat cac ranh gioi giua vung sang (50)
# va vung toi (10) thong qua cac gia tri lon.
```

```
28
```

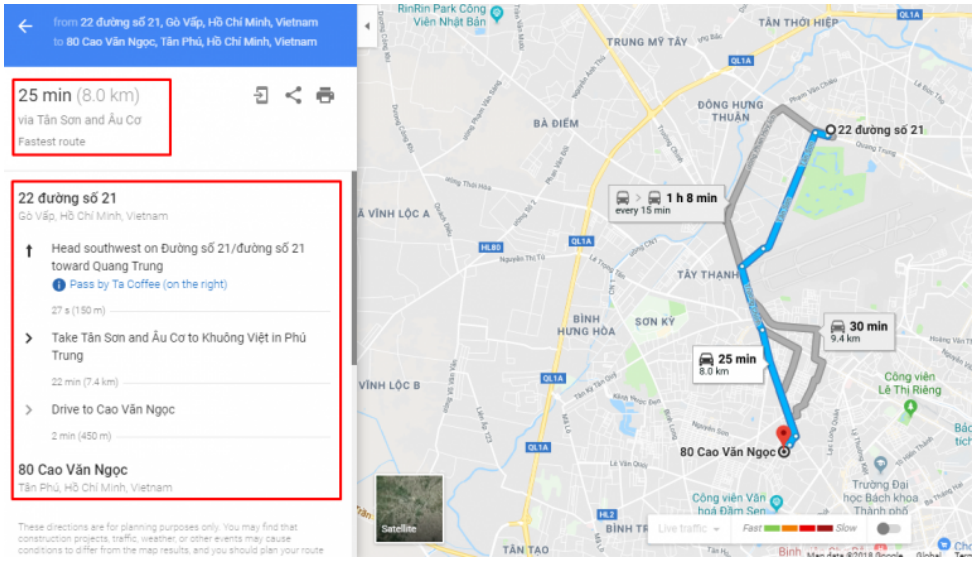
14.2 Lý thuyết đồ thị và hệ thống gợi ý

Biểu diễn đồ thị bằng ma trận kề

Đồ thị là một cấu trúc dữ liệu cực kỳ quan trọng trong công nghệ thông tin. Một cách hiệu quả để biểu diễn đồ thị cho máy tính hiểu là sử dụng **ma trận kề** A kích thước $n \times n$. Nếu Node i kết nối với Node j, phần tử $a_{ij} = 1$, ngược lại bằng 0.

Hệ thống gợi ý kết bạn (Facebook, Instagram...): Hãy coi người dùng là một đỉnh và quan hệ bạn bè là một cạnh nối. Khi đó, mạng xã hội là một ma trận khổng lồ.

Nếu bạn lấy **bình phương ma trận kề** (A^2), phần tử ở hàng i, cột j chính là **số lượng bạn chung** giữa người i và người j. Dựa vào con số này, Facebook sẽ ưu tiên gợi ý kết bạn cho những người có nhiều bạn chung nhất!



Hình 14.2: Minh họa thuật toán tìm đường đi ngắn nhất trên Google Maps

Ví dụ 14.2.1 Giả sử ta có một bản đồ khu vực gồm 4 địa điểm: Trạm A (Nhà), Trạm B (Quán Cafe), Trạm C (Trạm xăng), và Trạm D (Trường học). Khoảng cách (km) giữa các điểm được cho như sau:

- A nối với B (2 km), A nối với C (5 km).
- B nối với C (1 km), B nối với D (6 km).
- C nối với D (2 km).

Yêu cầu: Dùng ma trận trọng số và thuật toán Dijkstra để tìm khoảng cách ngắn nhất từ Nhà (A) đến Trường học (D).

```
import numpy as np
from scipy.sparse.csgraph import dijkstra

# 1. Lap Ma tran trong so (Weighted Adjacency Matrix)
# Cac hang/cot la: 0 (A), 1 (B), 2 (C), 3 (D)
# Gia tri 0 nghia la khong co duong di truc tiep
graph = np.array([
[0, 2, 5, 0],  # Tu A
[2, 0, 1, 6],  # Tu B
[5, 1, 0, 2],  # Tu C
[0, 6, 2, 0]   # Tu D
])

# 2. Chay thuat toan Dijkstra tu dinh nguon A (indices=0)
distances, predecessors = dijkstra(
csgraph=graph,
directed=False,
indices=0,
return_predecessors=True
)

# 3. In ket qua khoang cach toi dinh D (index=3)
print(f"Khoang cach ngan nhat tu Nha (A) den Truong (D) la: {distances[3]} km")

# Output: Khoang cach ngan nhat tu Nha (A) den Truong (D) la: 5.0 km
# (Lo trinh toi uu: A -> B -> C -> D = 2 + 1 + 2 = 5 km)

```

Mở rộng: Trong thực tế hệ thống của Google Maps hay Grab, ma trận này có kích thước lên đến hàng triệu hàng/cột (tương ứng với hàng triệu ngã tư). Hơn nữa, trọng số không chỉ là khoảng cách vật lý mà còn là **thời gian di chuyển**, được hệ thống liên tục cập nhật theo thời gian thực (kẹt xe, đèn đỏ) bằng các mô hình học máy để tính toán lại ma trận và đưa ra lộ trình mới ngay lập tức.

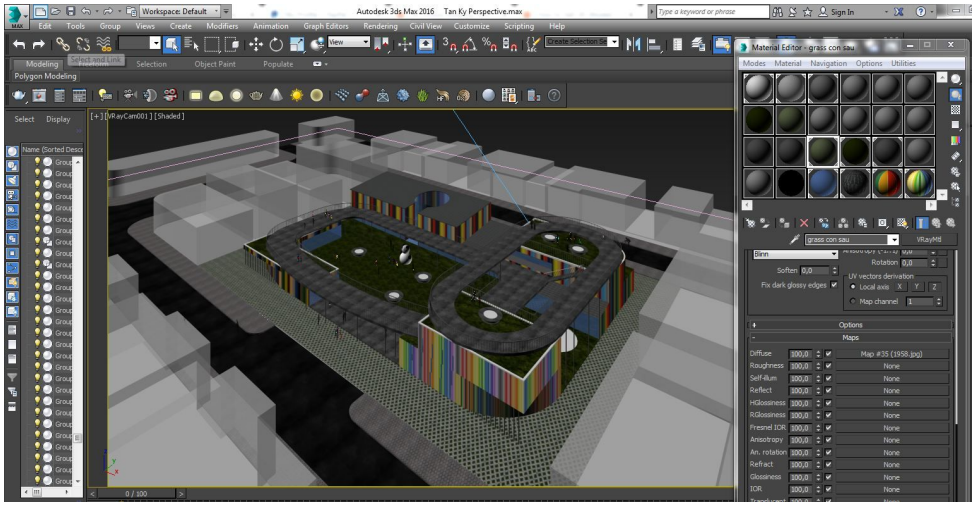
14.3 Giải tích trong đồ họa 3D và mô hình hóa

Đường cong Bezier và phương trình ánh sáng

Thế giới thực không phải là những khối vuông góc cạnh, mà là những bề mặt cong mượt mà. Để máy tính vẽ được những đường cong này (như nắp capo ô tô trong game đua xe), người ta dùng các phương trình tham số như **đường cong Bezier** hay **B-spline**.

Để tính toán tương phản ánh sáng và bóng đổ trên một bề mặt 3D cong, card đồ họa phải tính được vector pháp tuyến (vuông góc với bề mặt) tại mọi điểm ảnh. Vector pháp tuyến này được tìm ra bằng cách lấy **đạo hàm riêng** của phương trình bề mặt.

Để tạo ra hiệu ứng đổ bóng mềm hay tán xạ ánh sáng chân thực, máy tính phải giải **phương trình kết xuất** - bản chất là một phép **tích phân đa chiều** tính tổng mọi tia sáng dội vào điểm đó từ mọi hướng.



Hình 14.3: Thiết kế mô hình 3D bằng phần mềm Autodesk 3ds Max

Ví dụ 14.3.1 Vẽ một đường cong Bezier bậc 3 mượt mà đi qua các điểm điều khiển. Phương trình tham số của đường cong Bezier bậc 3 là một đa thức:

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t)t^2 P_2 + t^3 P_3 \quad \text{với } t \in [0,1]$$

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Dinh nghia 4 diem dieu khien (Control Points)
P0 = np.array([0, 0])   # Diem bat dau
P1 = np.array([1, 4])   # Diem uon 1
P2 = np.array([4, 5])   # Diem uon 2
P3 = np.array([5, 2])   # Diem ket thuc

# 2. Tao mang t chay tu 0 den 1 (dai dien cho thoi gian/tham so)
t = np.linspace(0, 1, 100)

# 3. Phuong trinh duong cong Bezier bac 3 (Ung dung toan lien tuc)
# Su dung broadcasting cua Numpy de tinh toan hang loat
B = ( (1-t)**3 )[:, None] * P0 + \
( 3 * (1-t)**2 * t )[:, None] * P1 + \
( 3 * (1-t) * t**2 )[:, None] * P2 + \
( t**3 )[:, None] * P3

# 4. Ve do thi (Phan nay may tinh se hien thi duong cong muot ma)
plt.plot(B[:, 0], B[:, 1], label="Bezier Curve", color='blue')
plt.scatter([P0[0], P1[0], P2[0], P3[0]],
[P0[1], P1[1], P2[1], P3[1]], color='red', zorder=5)
plt.legend()
plt.show()
```

14.4 Phương pháp số và giải thuật tính toán

Ứng dụng đạo hàm trong phương pháp Newton-Raphson

Máy tính không biết giải phương trình $x^2 - 2 = 0$ bằng cách chuyển vế lấy căn như con người. Thay vào đó, nó dùng **phương pháp số** để xấp xỉ nghiệm với độ chính xác cực cao.

Một trong những thuật toán kinh điển nhất là **phương pháp Newton-Raphson**. Nó sử dụng **đạo hàm** (đại diện cho tiếp tuyến của đồ thị) để trượt dần về phía nghiệm thực sự. Công thức truy hồi:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Ứng dụng: Hàm Math.sqrt() trong các ngôn ngữ lập trình C++, Java, Python được cài đặt dưới lõi phần cứng dựa trên biến thể của thuật toán này.

Ví dụ 14.4.1 Xây dựng hàm tính căn bậc hai của $N = 10$ (tức là giải phương trình $f(x) = x^2 - 10 = 0$) bằng phương pháp Newton mà không dùng thư viện toán học. Đạo hàm của hàm số là $f'(x) = 2x$.

```python
# Xay dung thuat toan Newton-Raphson tim can bac 2 cua N
def newton_sqrt(N, tolerance=1e-7):
# Buoc 1: Chon mot gia tri du doan ban dau x0
x = N / 2.0

iteration = 0
while True:
iteration += 1

# Buoc 2: Tinh f(x) va f'(x)
fx = x**2 - N
f_prime_x = 2 * x

# Buoc 3: Cap nhat x moi theo cong thuc Newton
x_new = x - (fx / f_prime_x)

# Kiem tra sai so (Tolerance)
if abs(x_new - x) < tolerance:
break

x = x_new
print(f"Vong lap {iteration}: x xap xi = {x:.6f}")

return x_new

# Tinh can bac 2 cua 10
result = newton_sqrt(10)
```

```
    print(f"\\nKet qua cuoi cung: {result:.6f}")

    # Output thuc te chung minh thuat toan hoi tu cuc nhanh:
    # Vong lap 1: x xap xi = 3.500000
    # Vong lap 2: x xap xi = 3.178571
    # Vong lap 3: x xap xi = 3.162319
    # Vong lap 4: x xap xi = 3.162278
    # Ket qua cuoi cung: 3.162278 (Chinh xac tuyet doi!)

```